CSE406 Project 2025

Attack Tools Implementation

Design Report

ARP Cache Poisoning + Man-in-the-Middle Attack Tool

Group Members:

Name	Student ID
Sheikh Rahat Mahmud	2005048
Tusher Bhomik	2005046

Course: CSE406 - Computer Security

Department of Computer Science and Engineering

Chittagong University of Engineering and Technology

Submission Date: Week 11, January 2025

Contents

1 Introduction

This design report presents a comprehensive approach to implementing an ARP (Address Resolution Protocol) cache poisoning attack combined with a man-in-the-middle (MITM) attack tool. The project aims to demonstrate the vulnerabilities inherent in the ARP protocol and how they can be exploited to intercept network communications in a controlled educational environment.

The tool will be implemented from scratch using Python, with custom packet crafting capabilities, avoiding the use of existing penetration testing frameworks. This approach ensures a deep understanding of the underlying protocols and attack mechanisms.

Disclaimer: This tool is developed solely for educational purposes and authorized security testing. It should never be used in unauthorized environments or for malicious activities.

2 Attack Definition and Network Topology

2.1 Attack Definition

ARP Cache Poisoning is a technique where an attacker sends falsified ARP messages onto a local area network. The goal is to associate the attacker's MAC address with the IP address of a legitimate computer or server on the network, causing any traffic meant for that IP address to be sent to the attacker instead.

Man-in-the-Middle (MITM) Attack is a form of eavesdropping where the attacker makes independent connections with the victims and relays messages between them, making them believe they are talking directly to each other over a private connection, when in fact the entire conversation is controlled by the attacker.

2.2 Network Topology

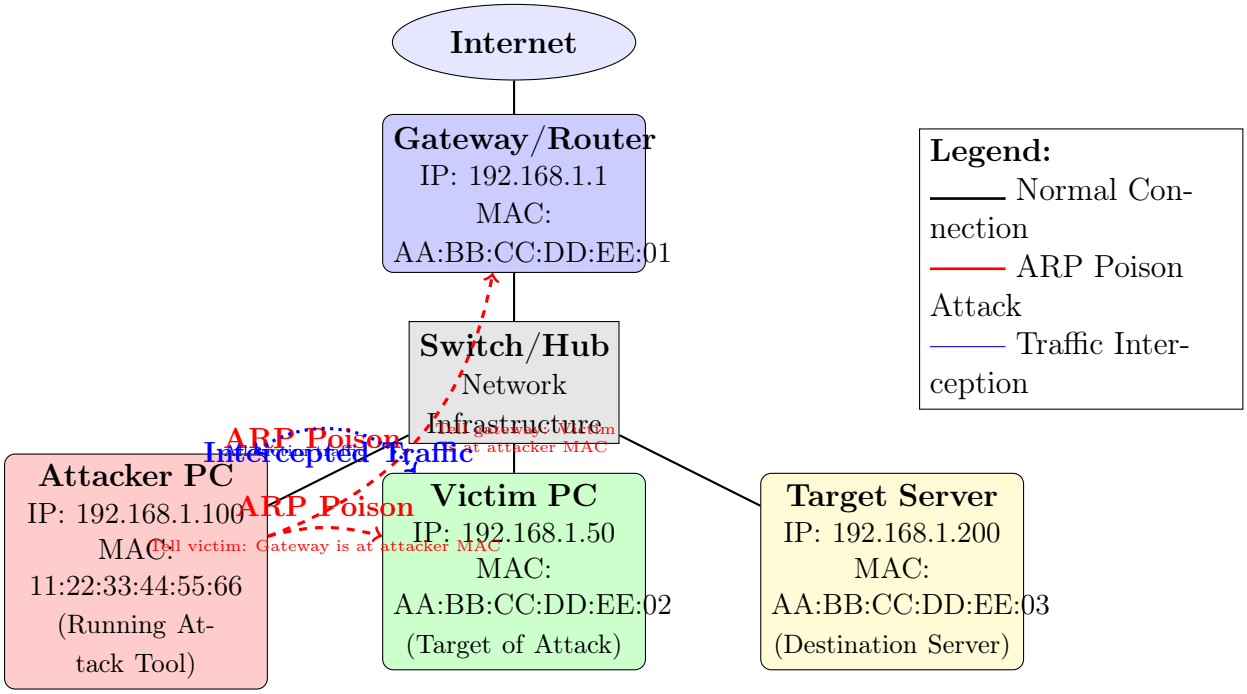

Figure 1: Improved Network Topology for ARP Cache Poisoning Attack

2.3 Network Components Description

- **Attacker PC:** The machine running our custom ARP poisoning tool (IP: 192.168.1.100)
- **Victim PC:** The target machine whose traffic will be intercepted (IP: 192.168.1.50)
- **Target Server:** The destination server that the victim is trying to communicate with (IP: 192.168.1.200)
- **Gateway/Router:** The network gateway providing internet access (IP: 192.168.1.1)
- **Switch/Hub:** Network infrastructure connecting all devices on the local segment

3 Timing Diagrams

3.1 Normal ARP Operation

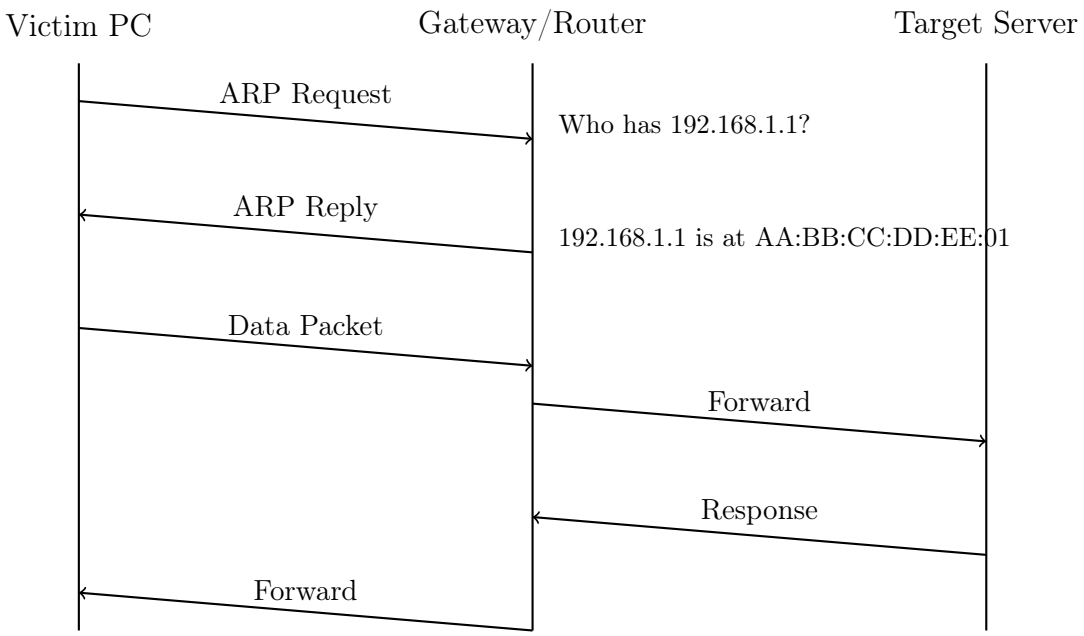

Figure 2: Normal ARP Operation Timing Diagram

3.2 ARP Cache Poisoning Attack Timing

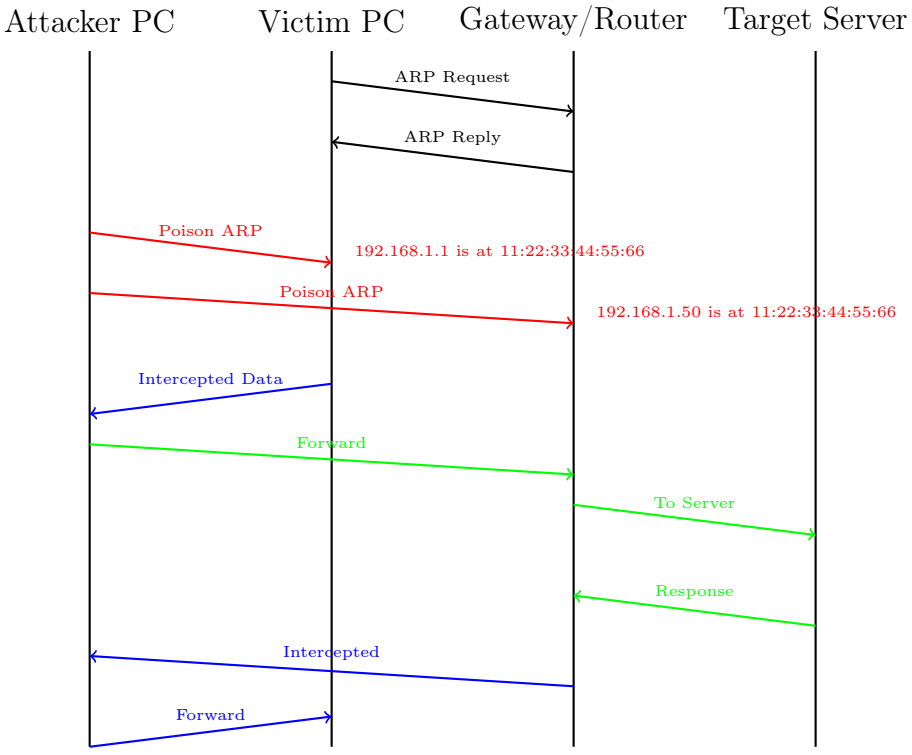

Figure 3: ARP Cache Poisoning Attack Timing Diagram

3.3 Attack Strategy Timeline

Table 1: Attack Phase Timeline

Phase	Duration	Activities
Network Discovery	0-10 seconds	Scan network for active hosts, identify gateway IP and MAC, select victim targets
Initial ARP Poisoning	10-15 seconds	Send malicious ARP replies to victim and gateway, verify ARP cache modification
Traffic Interception	15+ seconds	Start packet sniffing, intercept and log traffic, forward packets to maintain connectivity
Continuous Poisoning	Ongoing	Send poison packets every 2-3 seconds to maintain attack

4 Packet and Frame Details

4.1 Ethernet Frame Structure

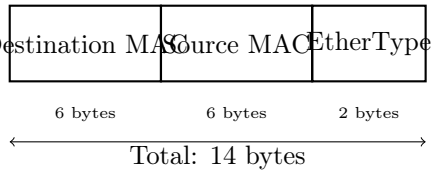

Figure 4: Ethernet Frame Header Structure

4.2 ARP Packet Structure

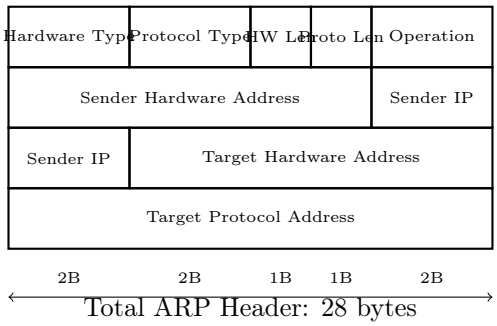

Figure 5: ARP Packet Header Structure

4.3 Packet Field Specifications

Table 2: ARP Packet Field Values

Field	Size	Normal Value	Attack Value
Hardware Type	2 bytes	0x0001 (Ethernet)	0x0001 (Ethernet)
Protocol Type	2 bytes	0x0800 (IPv4)	0x0800 (IPv4)
Hardware Length	1 byte	0x06	0x06
Protocol Length	1 byte	0x04	0x04
Operation	2 bytes	0x0002 (Reply)	0x0002 (Reply)
Sender MAC	6 bytes	Gateway MAC	Attacker MAC
Sender IP	4 bytes	Gateway IP	Gateway IP (Spoofed)
Target MAC	6 bytes	Victim MAC	Victim MAC
Target IP	4 bytes	Victim IP	Victim IP

4.4 Header Modifications for Attack

The key modifications made to create malicious ARP packets are:

1. **Sender MAC Address:** Changed from legitimate gateway MAC to attacker's MAC address
2. **Source MAC in Ethernet Header:** Modified to attacker's MAC address
3. **Gratuitous ARP:** Using unsolicited ARP replies to poison the cache
4. **Bidirectional Poisoning:** Creating separate packets for victim→gateway and gateway→victim poisoning

```
# Poison packet to victim (claiming to be gateway)
ethernet_dst = victim_mac           # Target: victim
ethernet_src = attacker_mac         # Source: attacker
arp_sender_mac = attacker_mac       # SPOOFED: claiming attacker is
    gateway
arp_sender_ip = gateway_ip          # SPOOFED: gateway's IP
arp_target_mac = victim_mac         # Victim's actual MAC
arp_target_ip = victim_ip           # Victim's actual IP
```

Listing 1: Packet Crafting Example

5 Design Justification

5.1 Technical Foundation

Our design leverages fundamental weaknesses in the ARP protocol:

- **No Authentication:** ARP has no built-in mechanism to verify the authenticity of ARP messages

- **Stateless Protocol:** ARP replies are accepted regardless of whether requests were made
- **Cache Update Policy:** Most systems automatically update their ARP cache upon receiving ARP replies
- **Trust Model:** Local network devices implicitly trust ARP messages from other devices

5.2 Attack Effectiveness Factors

5.2.1 ARP Cache Poisoning Success

- **Gratuitous ARP Replies:** Sending unsolicited ARP replies forces immediate cache updates
- **Continuous Poisoning:** Regular packet transmission maintains the poisoned state
- **Bidirectional Attack:** Poisoning both victim and gateway ensures complete traffic redirection
- **Timing Consideration:** ARP cache entries typically expire in 2-20 minutes, requiring periodic refresh

5.2.2 Traffic Interception Success

- **IP Forwarding:** Enabling IP forwarding allows transparent packet forwarding to maintain connectivity
- **Layer 2 Operation:** Operating at the data link layer provides complete traffic visibility
- **Protocol Agnostic:** Can intercept any protocol running over IP (HTTP, HTTPS, FTP, etc.)
- **Stealth Operation:** Victims experience minimal service disruption when properly implemented

5.3 Expected Success Conditions

5.3.1 Network Requirements

- Same broadcast domain as target devices
- Switched Ethernet environment (most common)
- No advanced security measures (Dynamic ARP Inspection, etc.)
- Standard ARP implementation on target systems

5.3.2 System Requirements

- Raw socket access for custom packet crafting
- Administrative privileges for network interface control
- Sufficient processing power for real-time packet handling
- Network interface supporting promiscuous mode

5.4 Potential Limitations

5.4.1 Technical Limitations

1. **Static ARP Entries:** Pre-configured static entries cannot be poisoned
2. **ARP Inspection:** Advanced switches may validate ARP packet consistency
3. **Detection Systems:** IDS/IPS systems may identify unusual ARP traffic patterns
4. **Network Segmentation:** VLANs and proper network isolation reduce attack surface

5.4.2 Implementation Challenges

1. **Timing Sensitivity:** Balance between stealth and attack effectiveness
2. **Packet Forwarding:** Maintaining connection transparency while intercepting data
3. **Multi-target Coordination:** Managing multiple simultaneous attack sessions
4. **Error Handling:** Graceful handling of network topology changes and failures

5.5 Design Advantages

5.5.1 Custom Implementation Benefits

- **Educational Value:** Complete understanding of underlying protocol operations
- **Customization:** Tailored features for specific attack scenarios and requirements
- **Stealth Capabilities:** Avoiding detection signatures associated with known tools
- **Performance Optimization:** Optimized specifically for target network conditions

5.5.2 Architecture Benefits

- **Modular Design:** Separate, maintainable components for discovery, poisoning, and interception
- **Scalability:** Framework supports multiple simultaneous targets and attack vectors
- **Extensibility:** Foundation for implementing additional MITM attack types
- **Portability:** Cross-platform implementation using standard Python libraries

6 Implementation Strategy

6.1 Programming Language Selection

We have chosen **Python 3.x** for implementation based on the following justifications:

- **Raw Socket Support:** Native support for low-level network programming
- **Binary Data Handling:** Struct module provides excellent packet construction capabilities
- **Cross-platform Compatibility:** Runs on Linux, Windows, and macOS
- **Development Speed:** Enables rapid prototyping and debugging
- **Educational Clarity:** Clean, readable code suitable for academic purposes

6.2 Core Component Architecture

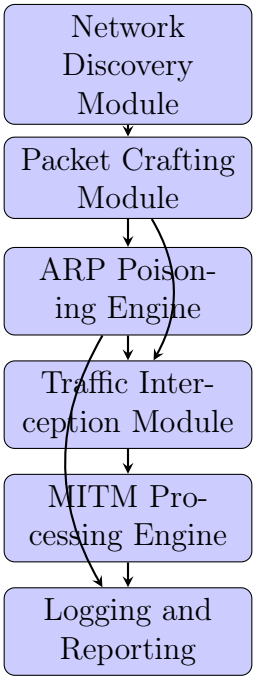

Figure 6: System Architecture Overview

6.3 Implementation Phases

Table 3: Implementation Phase Plan

Phase	**Duration**	**Deliverables**
Foundation	Week 12 (Days 1-3)	Raw socket framework, basic packet construction, network interface enumeration
Core Attack	Week 12 (Days 4-7)	ARP packet crafting, traffic interception, bidirectional poisoning logic
Enhancement	Week 13 (Days 1-4)	Multi-target support, advanced filtering, comprehensive logging
Testing & Demo	Week 13 (Days 5-7)	Integration testing, demonstration preparation, final documentation

7 Group Member Contributions

7.1 Planned Responsibilities

Table 4: Group Member Responsibility Matrix

Component	**Sheikh Rahat Mahmud (2005048)**	**Tusher Bhomik (2005046)**
Network Discovery	Primary Developer	Code Review & Testing
Packet Crafting	Code Review & Testing	Primary Developer
ARP Poisoning Engine	Primary Developer	Code Review & Testing
Traffic Interception	Code Review & Testing	Primary Developer
MITM Processing	Primary Developer	Code Review & Testing
Logging & Reporting	Code Review & Testing	Primary Developer
Integration & Testing	Shared Responsibility	Shared Responsibility
Documentation	Shared Responsibility	Shared Responsibility
Demo Preparation	Shared Responsibility	Shared Responsibility
Defense Mechanisms	Shared Responsibility	Shared Responsibility

8 Conclusion

This design report presents a comprehensive approach to implementing an ARP cache poisoning and man-in-the-middle attack tool. Our design leverages fundamental weaknesses in the ARP protocol while providing valuable educational insights into network security vulnerabilities.

The proposed modular architecture ensures maintainability and extensibility while meeting all academic requirements for custom tool development. The implementation strategy balances educational value with practical functionality, providing a solid foundation for understanding both attack techniques and defensive measures.

8.1 Key Design Strengths

- **Educational Focus:** Comprehensive learning of network protocols and security concepts
- **Technical Depth:** Detailed understanding of packet-level network operations
- **Practical Implementation:** Real-world applicable attack demonstration
- **Modular Architecture:** Clean, maintainable code structure
- **Custom Development:** Original implementation meeting academic requirements

8.2 Next Steps

Following this design phase, our group will proceed with the implementation phase during weeks 12-13, focusing on:

1. Setting up the development and testing environment
2. Implementing core packet crafting and ARP poisoning functionality
3. Developing traffic interception and analysis capabilities
4. Conducting comprehensive testing in controlled lab environments
5. Preparing for the final demonstration and report submission

The successful completion of this project will demonstrate not only the technical implementation of network attacks but also the importance of proper network security measures and the need for ongoing vigilance in network administration.

Appendices

Appendix A: ARP Protocol Specification (RFC 826)

The Address Resolution Protocol (ARP) is defined in RFC 826 and operates at the Network Interface Layer of the TCP/IP protocol stack. Key specifications include:

- Protocol operates on broadcast medium (Ethernet)
- Maps 32-bit IP addresses to 48-bit MAC addresses
- Uses broadcast for requests, unicast for replies
- No authentication or verification mechanisms
- Cache timeout typically 2-20 minutes depending on implementation

Appendix B: Ethical Considerations

This project is developed under strict ethical guidelines:

- Educational purpose only - no malicious intent
- Authorized testing environments exclusively
- Responsible disclosure of any discovered vulnerabilities
- Proper data handling and privacy protection
- Compliance with university and legal requirements

Appendix C: Legal Disclaimer

The tools and techniques described in this report are for educational and authorized security testing purposes only. Unauthorized access to computer networks is illegal and unethical. Users of this information must ensure they have proper authorization before conducting any security testing.